

Malware Analysis A3

Objective:

Task 1: File Analysis Using PESTudio and PEId

Entropy Analysis: Use PESTudio and PEId to analyze the entropy of the .exe file. Identify regions of the file with high entropy, which may indicate packed sections. Explain the significance of entropy in detecting packed files.

Use the tools to determine if the .exe file is packed or unpacked. What methods can you use to confirm if the file is packed? Provide at least two indicators. Discuss the significance of packing in executable files.

List the sections of the .exe file, such as .text, .data, .rdata, etc. For each section, provide information on its size, characteristics (read/write/execute permissions), and any suspicious findings. Are there any unusual sections present? If so, explain their possible purpose.

Identify the compiler stamp or timestamp present in the file. What information can the compiler stamp reveal about the file's creation? What implications does the presence of a specific compiler have on the file's potential origin?

List the libraries, APIs and functions that are imported by the file. What are the most important imports related to system functionality (e.g., kernel32.dll, user32.dll)? Check for any exported functions. Describe the significance of the functions being exported and any security implications of exposing those functions. Discuss how imports and exports contribute to the file's functionality and potential risks.

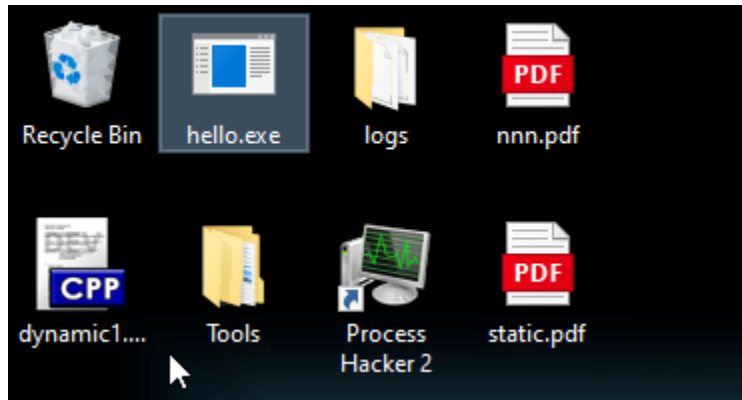
Task 2: File Packing and Unpacking

Use a packing tool (such as UPX or similar) to pack the .exe file. Describe the changes that occurred in the file's structure after packing, particularly focusing on section changes, entropy, and size. Provide a before-and-after comparison of the file's characteristics.

Once packed, attempt to unpack the file using unpacking tools or techniques. Explain the steps you took to unpack the file. After unpacking, perform the same analysis on the file as in Task 1 (entropy, section details, imports/exports). Compare these findings with the original unpacked file.

Entropy Analysis

Targeted file - hello.exe



pestudio 9.58 - Malware Initial Assessment - www.winitor.com (read-only)

file settings about

c:\users\hrp\desktop\hello.exe

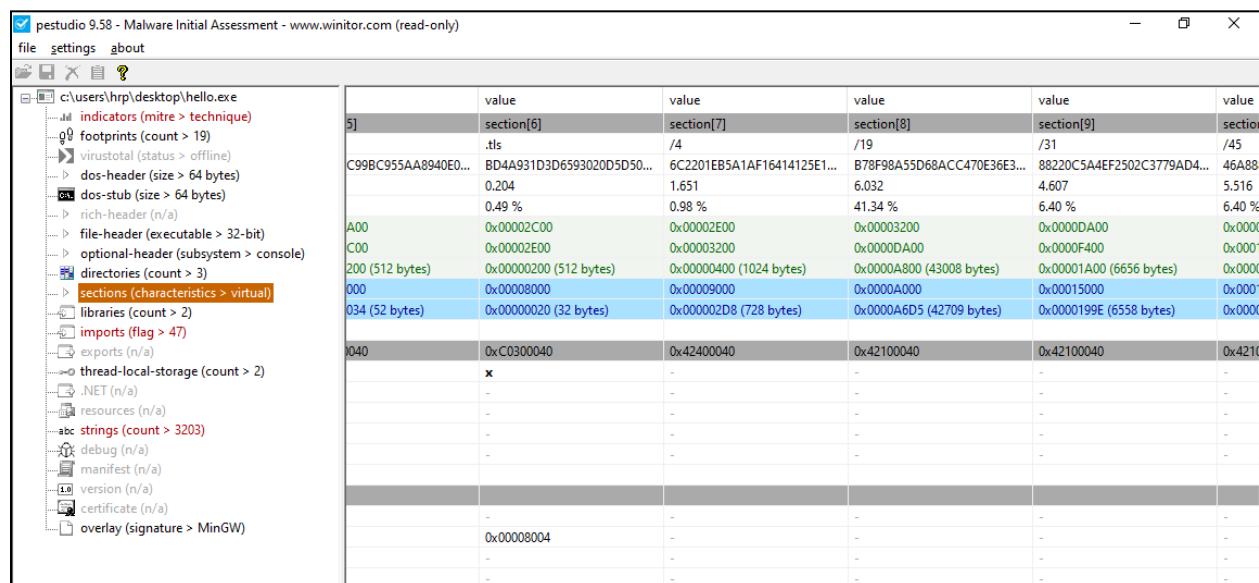
- indicators (mitre > technique)
- footprints (count > 19)
- virusotal (status > offline)
- dos-header (size > 64 bytes)
- dos-stub (size > 64 bytes)
- rich-header (n/a)
- file-header (executable > 32-bit)
- optional-header (subsystem > console)
- directories (count > 3)
- sections (characteristics > virtual)
- libraries (count > 2)
- imports (flag > 47)
- exports (n/a)
- thread-local-storage (count > 2)
- .NET (n/a)
- resources (n/a)
- strings (count > 3203)
- debug (n/a)
- manifest (n/a)
- version (n/a)
- certificate (n/a)
- overlay (signature > MinGW)

property	value
footprint > sha256	701DADBEE86DD68CAC9CFE4D3AFDB1D45411364038B488FD420701052451D2EE
first-bytes-hex	4D 5A 90 00 03 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 40 00 00 00 00 00 00 00 00
first-bytes-text	M Z @
file > size	104045 bytes
entropy	5.407
signature	n/a
tooling	MinGW
file-type	executable
cpu	32-bit
subsystem	console
file-version	n/a
description	n/a
stamps	
compiler-stamp	Tue Oct 15 06:00:09 2024 UTC
debug-stamp	n/a
resource-stamp	n/a
import-stamp	n/a
export-stamp	n/a
names	
file	c:\users\hrp\desktop\hello.exe
debug	n/a
export	n/a
version	n/a
manifest	n/a
.NET > module	n/a
certificate > program-name	n/a

An entropy score of **5.407** (on a scale of 0 to 8) indicates a **moderate level of randomness** within the hello.exe file. It doesn't necessarily mean "random files have been added," but rather that the data itself is less structured than typical program code.

A score in this range suggests that the file might be **compressed** or **lightly packed**. While not as high as heavily encrypted malware (which often scores >7.0), it's more random than a standard, unpacked executable (which typically scores between 4.8 and 5.2).

- **Obfuscate** (hide) the true code from antivirus scanners.
- Make reverse engineering more difficult.
- **Low Entropy:** Typical for simple programs or files with lots of repetitive, uninitialized data.
- **High Entropy:** Common in compressed files (.zip, .rar), encrypted data, and packed malware.



Entropy is a measure of randomness or disorder in data, typically rated on a scale from 0 (perfectly ordered) to 8 (completely random).

- In an **unpacked file**, different sections have distinct and predictable entropy levels. For example, the `.text` section containing code has medium entropy, while a `.bss` section with uninitialized data has very low entropy.
- In a **packed file**, the original code is compressed or encrypted. This process makes the data appear highly random. As a result, packed files usually have at least one large section with **very high entropy**, often greater than 7.0.

Name	V. Offset	V. Size	R. Offset	R. Size	Flags
.text	00001000	00001680	00000400	00001800	60500020
.data	00003000	0000002C	00001C00	00000200	C0300040
.rdata	00004000	00000460	00001E00	00000600	40300040
.bss	00005000	00000450	00000000	00000000	C0700080
.idata	00006000	00000568	00002400	00000600	C0300040
.CRT	00007000	00000034	00002A00	00000200	C0300040
.tls	00008000	00000020	00002C00	00000200	C0300040
/4	00009000	000002D8	00002E00	00000400	42400040

In a packed file, we can see UPX0, UPX1, and UPX2.

Name	V. Offset	V. Size	R. Offset	R. Size	Flags
UPX0	00001000	00017000	00000200	00000000	E0000080
UPX1	00018000	00007000	00000200	00006A00	E0000040
UPX2	0001F000	00001000	00006C00	00000200	C0000040

The next step is the section analysis. We observed fluctuations in entropy between packed and unpacked files. Here are the entropy details for the packed file: the entropy in section [1] is approximately 7.8.

pestudio 9.58 - Malware Initial Assessment - www.winitor.com (read-only)

file settings about

c:\users\hrp\desktop\new.exe

indicators (sections > writable)

footprints (count > 7)

virustotal (status > offline)

dos-header (size > 64 bytes)

dos-stub (size > 64 bytes)

rich-header (n/a)

file-header (executable > 32-bit)

optional-header (subsystem > console)

directories (count > 2)

sections (characteristics > self-modifying)

libraries (count > 2)

imports (flag > 5)

property	value	value	value
section	section[0]	section[1]	section[2]
name	UPX0	UPX1	UPX2
footprint > sha256	n/a	4429CEE5240EA2B1DA62D92...	108CC47CE6E1A8C05C97FC...
entropy	n/a	7.817	1.708
file-ratio (49.90%)	n/a	48.98 %	0.92 %
raw-address (begin)	0x00000200	0x00000200	0x00006C00
raw-address (end)	0x00000200	0x00006C00	0x00006E00
raw-size (27648 bytes)	0x00000000 (0 bytes)	0x00006A00 (27136 bytes)	0x00000200 (512 bytes)
virtual-address	0x00001000	0x00018000	0x0001F000
virtual-size (126976 bytes)	0x00017000 (94208 bytes)	0x00007000 (28672 bytes)	0x00001000 (4096 bytes)

Section Details

The executable has four primary sections: .text, .data, an unnamed section containing import data (likely .rdata), and .bss.

pestudio 9.58 - Malware Initial Assessment - www.winitor.com (read-only)

file settings about

c:\users\hrp\desktop\hello.exe

indicators (mitre > technique)

footprints (count > 19)

virustotal (status > error)

dos-header (size > 64 bytes)

dos-stub (size > 64 bytes)

rich-header (n/a)

file-header (executable > 32-bit)

optional-header (subsystem > console)

directories (count > 3)

sections (characteristics > virtual)

libraries (count > 2)

imports (flag > 47)

property	value	value	value	value
section	section[0]	section[1]	section[2]	section[3]
name	.text	.data	.rdata	.bss
footprint > sha256	C735B475B1780438B91F5FE...	68BF6796BB27605D29A4ECF...	25686C3D4A4CCE2F48C586...	n/a
entropy	5.734	0.536	4.178	n/a
file-ratio (72.83%)	5.91 %	0.49 %	1.48 %	n/a
raw-address (begin)	0x00000400	0x00001C00	0x00001E00	0x00000000
raw-address (end)	0x00001C00	0x00001E00	0x00002400	0x00000000
raw-size (75776 bytes)	0x00001800 (6144 bytes)	0x00000200 (512 bytes)	0x00000600 (1536 bytes)	0x00000000 (0 bytes)
virtual-address	0x00001000	0x00003000	0x00004000	0x00005000
virtual-size (72470 bytes)	0x00001680 (5760 bytes)	0x0000002C (44 bytes)	0x00000460 (1120 bytes)	0x00000450 (1104 bytes)

1. .text (section[0])

- Size: 5760 bytes (Virtual Size)
- Characteristics: Execute. This section contains the executable code, so this permission is expected.
- Suspicious Findings: This section has a high entropy of 5.734. Entropy measures randomness; a high value (typically > 5.5) can indicate that the code is packed or encrypted. This is a common technique used by malware to hide its true nature from antivirus software and analysts.

2. .data (section[1])

- Size: 44 bytes (Virtual Size)
- Characteristics: Write. This section stores initialized global and static variables, so write permission is normal.
- Suspicious Findings: No suspicious findings are noted for this section.

3. Unnamed Section (section[2])

- Size: 1120 bytes (Virtual Size)
- Characteristics: Read-only (not writable). This section contains the import directory (directory->import), which lists the functions the program uses from other libraries (like DLLs). It is typically named .rdata (read-only data).
- Suspicious Findings: No suspicious findings are noted for this section.

Unusual and Suspicious Section

.bss (section[3])

- Size: 1104 bytes (Virtual Size)
- Characteristics: Write and Execute.
- Suspicious Findings: The combination of write (W) and execute (X) permissions is highly suspicious and a significant red flag.

Compiler Stamp Information

The compiler stamp, or timestamp, identified in the file is:

- Value: \$0x670E04E9\$
- Timestamp: Tue Oct 15 06:00:09 2024 | UTC

general		
compiler-stamp	0x670E04E9	Tue Oct 15 06:00:09 2024 UTC
size-of-optional-header	0x00E0	224 bytes
signature	0x00004550	PE00
machine	0x014C	Intel-386
sections-count	0x000F	15
pointer-symbol-table	0x00012C00	0x00000000 (expected)
number-of-symbols	0x000004E7	0x00000000 (expected)

This timestamp is embedded into the executable's header by the compiler during the compilation process. It indicates the precise moment the file was created. This information can be crucial for establishing a timeline during a digital forensics or malware analysis investigation. However, it's important to be aware that an attacker can deliberately alter this timestamp in a technique known as timestomping to mislead investigators.

Compiler and Origin Implications

The `pestudio` analysis also identifies the compiler used to create the file:

- **Compiler Signature:** MinGW (Minimalist GNU for Windows)

The use of the MinGW compiler has several implications for the file's potential origin:

- **Open-Source Toolchain:** MinGW is a free and open-source compiler suite that allows developers to create native Windows applications using GNU tools.
- **Widespread Use:** It is widely used by both legitimate software developers and malware authors due to its accessibility and lack of licensing costs.
- **Attribution Difficulty:** Because it is freely available and doesn't require registration, its use can make it more difficult to trace the software back to a specific commercial entity or developer, a feature often exploited by malicious actors.

Imported Libraries

The executable imports a total of 47 functions from two standard Windows libraries:

1. **KERNEL32.dll**: Imports 19 functions.
2. **msvcrt.dll**: Imports 28 functions.

While the specific function names aren't visible in this view, the libraries themselves tell us a lot about the file's foundation.

The screenshot shows the Malware Analysis tool interface. The top bar displays the file path: `c:\users\hrp\desktop\hello.exe`. The left sidebar lists various file properties, including indicators, footprints, status, headers, stubs, sections, libraries, imports, exports, thread-local storage, .NET, resources, strings, debug, manifest, version, certificate, and overlay. The main pane displays a table of properties for the selected file.

library (2)	duplicate (0)	flag (0)	first-thunk-original (INT)	first-thunk (IAT)	type (1)	imports (47)	group
KERNEL32.dll	-	-	0x0000603C	0x00006100	implicit	19	-
msvcrt.dll	-	-	0x0000608C	0x00006150	implicit	28	-

What the File Borrows (Imports)

This file borrows tools from two main "toolboxes" (libraries) that are part of Windows:

1. **KERNEL32.dll**: This is the most important toolbox. It contains the essential tools for any program to run on Windows, like managing memory, handling files, and running processes.
2. **msvcrt.dll**: This toolbox provides common, basic functions, like tools to print text on the screen.

What are the File Shares (Exports)

The analysis shows this file **does not share any of its tools**.

- An "export" is when a program makes its functions available for other programs to use.
- Since this file has no exports, it's a **user, not a sharer**. It's designed to run on its own and doesn't offer any services to other software.

What This Means in Simple Terms

- **Functionality**: The file works by using standard, built-in Windows tools.
- **Risk**: Security analysts look at what tools a file borrows to see what it's trying to do. If it borrowed tools for networking or deleting files, that could be a red flag.
- **Security**: Because this file doesn't share any functions (no exports), it has fewer "open doors" that another malicious program could use to attack it. It keeps to itself.

Task 2

Converting an unpacked file to a packed file

Commands - upx -o <newfilename> <file-to-be-converted>

```
FLARE-VM 12-08-2025 12:23:10.83
C:\Users\hrp\Desktop>upx -o new.exe hello.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

      File size      Ratio      Format      Name
      -----
      104045 ->      55405      53.25%      win32/pe      new.exe

                                                                 Packed 1 file.

FLARE-VM 12-08-2025 12:23:27.98
C:\Users\hrp\Desktop>
```

The core principle of a packer like UPX is to **compress** the original executable's code and data and bundle it with a small piece of code called a **decompression stub**. When you run the

packed file, the stub executes first, decompresses the original program into memory, and then transfers control to it.

Comparison of Packed and Unpacked Files

Characteristic	Before Packing (<i>hello.exe</i>)	After Packing (<i>new.exe</i>)
File Size	Larger (104,045 bytes)	Significantly smaller (55,405 bytes)
Number of Sections	Typically 3-5+ sections	Typically 2-3 sections
Section Names	Standard names like .text, .rdata, .data	Packer-specific names like UPX0, UPX1
Entropy	Moderate; varies by section	Very high and uniform across packed sections

Unpacking the packed file

Command used - `upx -d <packed-filename>`

- `upx`: The name of the executable.
- `-d`: The flag for **decompress** or unpack.
- `new.exe`: The target file to be unpacked.

```
FLARE-VM 12-08-2025 12:23:27.98
C:\Users\hrp\Desktop>upx -d new.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

      File size      Ratio      Format      Name
      -----
      104045 <-      55405      53.25%      win32/pe      new.exe

Unpacked 1 file.

FLARE-VM 12-08-2025 12:33:04.83
C:\Users\hrp\Desktop>_
```

Analysis: Original vs. Unpacked File

Characteristic	Original File (hello.exe)	Unpacked File (new.exe)	Comparison
File Size	104,045 bytes	104,045 bytes	Identical
Section Names	.text, .rdata, .data...	.text, .rdata, .data...	Identical
Entropy	Moderate, varied by section	Moderate, varied by section	Identical
Import Table	Fully visible and readable	Fully visible and readable	Identical

Conclusion

- **Size & Sections:** The file size returned to its exact original value. The UPX0 and UPX1 sections are gone, and the standard sections like **.text** and **.data** have been fully restored. **Entropy:** The high entropy of the packed state has vanished. The unpacked file's entropy profile is once again moderate and varies from section to section, reflecting the difference between structured code and data. This is because the data is no longer compressed.
- **Imports:** This is a crucial restoration. While the packed file hid its true dependencies, the unpacked file now has a **fully restored Import Address Table (IAT)**. We can see all the functions and libraries (like `kernel32.dll` or `user32.dll`) that the program needs to run. This is one of the main goals of unpacking during malware analysis—to reveal what the program does.